



Predicting What Walmart Sells Tomorrow

28-Day Demand Forecasting on the M5 Dataset

The Forecasting Challenge

Situation

Ten stores. Around 3,000 products in each. Every one of them needs a stocking decision, every single day.

That is 30,490 separate forecasts to produce, and then produce again tomorrow.

Why it is hard

- **Demand swings** sharply between departments and days
- **Most days are zero** — a typical item sells nothing in a typical store
- **Mistakes cost both ways** — overstock ties up cash, stockouts lose the sale

The question: can a model forecast 28 days ahead well enough to guide real stocking decisions — and how would we know if it did?

Context

Data

Solution

Implementation

The Data

30,490

item-store series

1,941

days of history

26.5M

rows after reshaping

68%

of item-days are zero

What we get

- **Sales** — daily units, Jan 2011 to Apr 2016
- **Prices** — weekly, per store and item
- **Calendar** — holidays, SNAP benefit days, weekends

How it nests

3 states → 10 stores → 3 categories

→ 7 departments → 3,049 items

Forecasts are judged at every one of these levels, not just the bottom.

Context

Data

Solution

Implementation

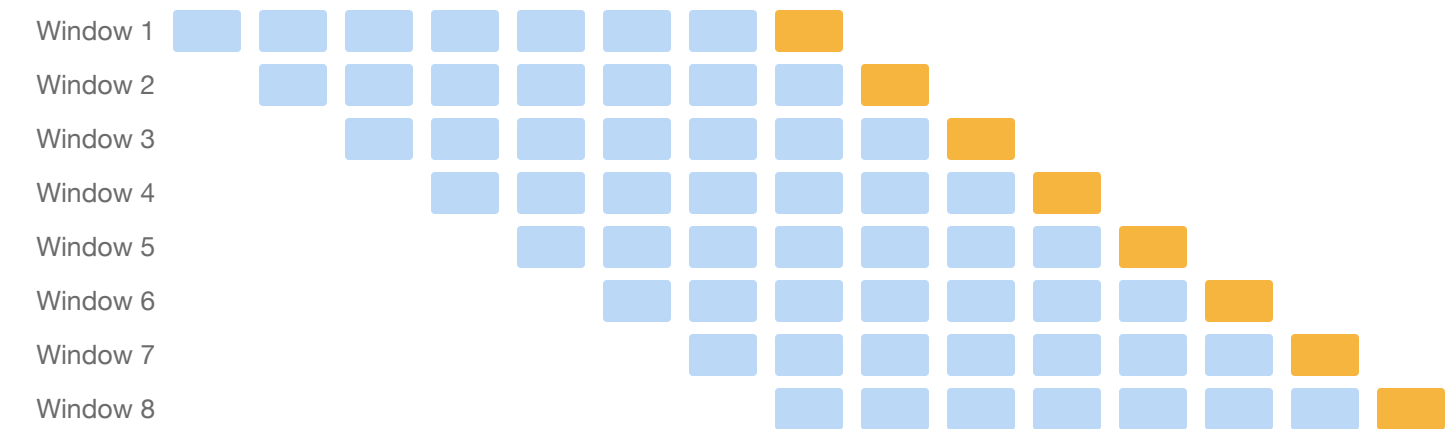
Four Models, Increasing Complexity

Model	Idea	What happened
LightGBM	Boosted trees on engineered features	Broke on a data leak — see slide 8
LSTM	Sequence model, predicts one day at a time	Trains slowly, no better for it
GRU	Lighter sequence model, same approach	Best model built — faster and more accurate than LSTM
Seq2Seq GRU	Encoder–decoder, all 28 days at once	Quickest to train, slightly behind GRU

Takeaway: GRU won among the models — but winning the comparison turned out not to be the same as being useful.

How the Model Learns

days it reads day it predicts



The network reads a fixed window of recent days and predicts the next one. Slide the window forward and repeat — that is the whole training set. To forecast 28 days, its own prediction is fed back in as input, 28 times.

This matters later: feeding predictions back into the model is exactly where the forecast goes wrong.

Not All Mistakes Matter Equally

Missing a fast seller by ten units is a real problem. Missing a slow one by ten units barely registers.

WRMSSE

The competition metric weights each error by the dollars that product actually sells, and scores it against a naive “same as yesterday” forecast.

Below 1.0 means you beat that naive forecast.

Above 1.0 means you did not.

The baseline, set first

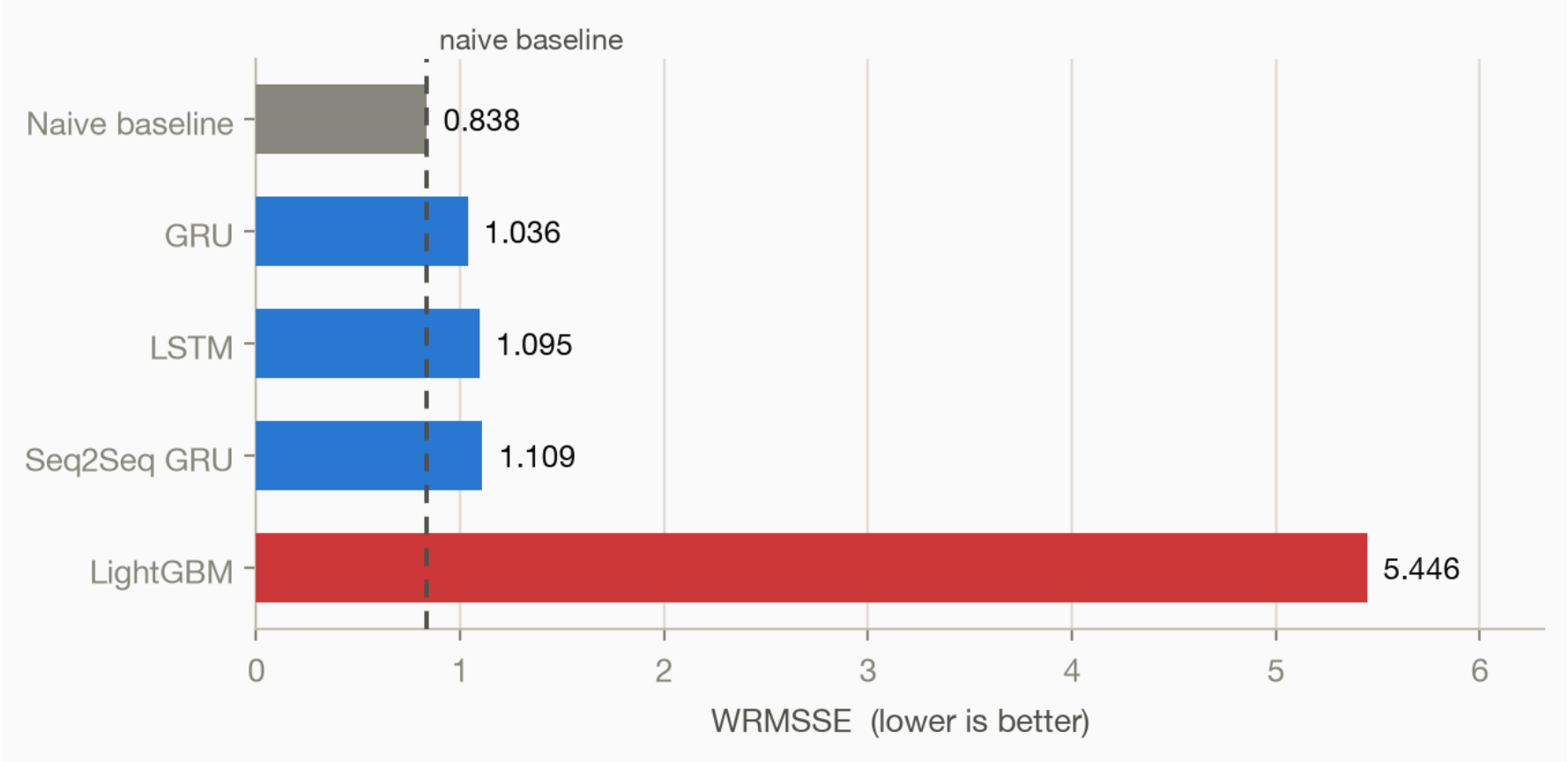
Before training anything, we scored the simplest possible forecast: repeat the last 28 days.

0.8377

naive baseline — the number to beat

Why set it first: without a baseline, any score sounds reasonable. With one, you know immediately whether the model earned its keep.

Results



WRMSSE on the 28 held-out days. Lower is better. Dashed line is the naive baseline.

No model beat the baseline. The best of them, the GRU at 1.0365, is 24% worse than simply repeating last month's sales.

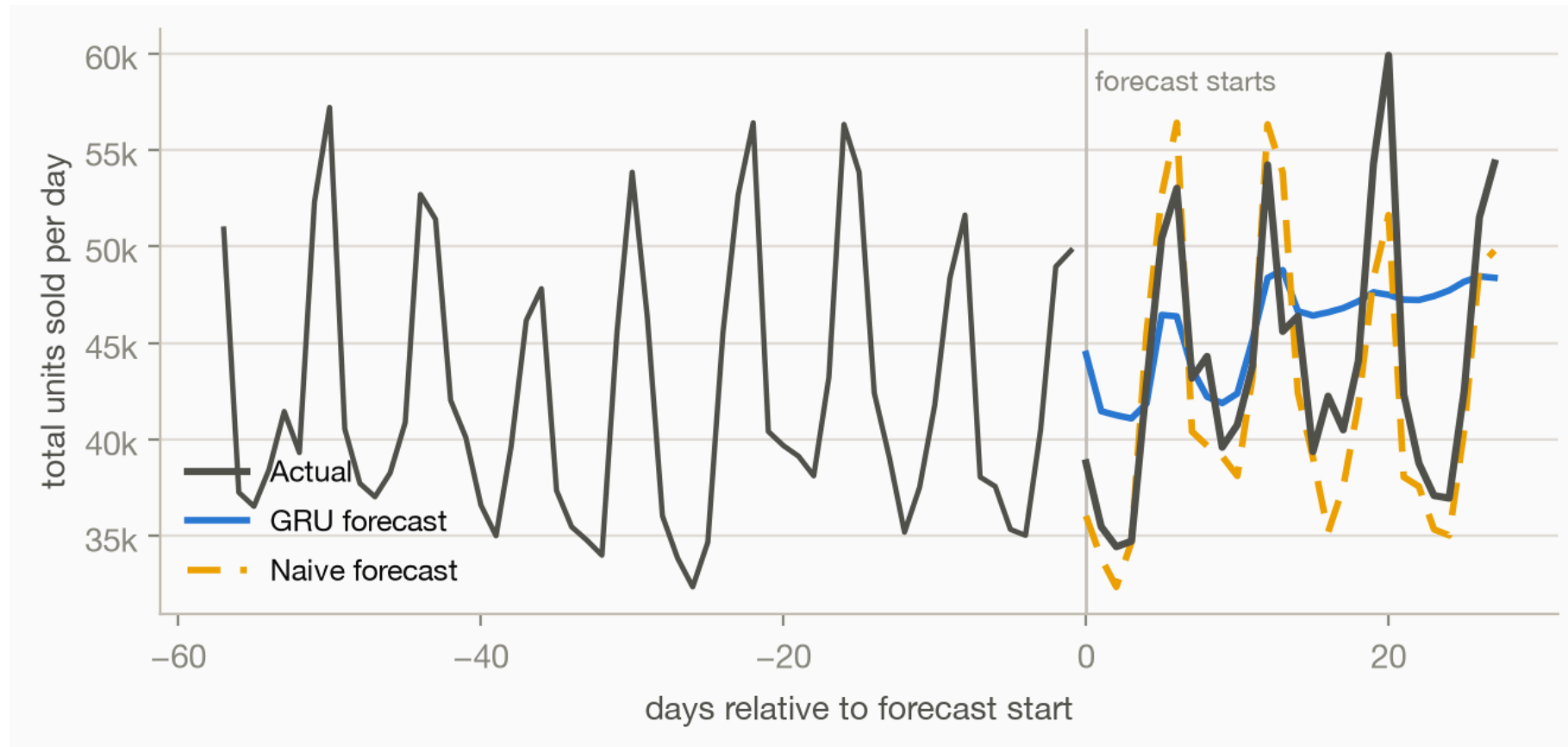
Context

Data

Solution

Implementation

Why: The Forecast Goes Flat



Total units sold per day. History on the left, the 28-day forecast on the right.

Sales run on a weekly cycle. The naive forecast copies it exactly. The GRU settles into a flat line near the average — it learns the level of demand and loses the shape.

Context

Data

Solution

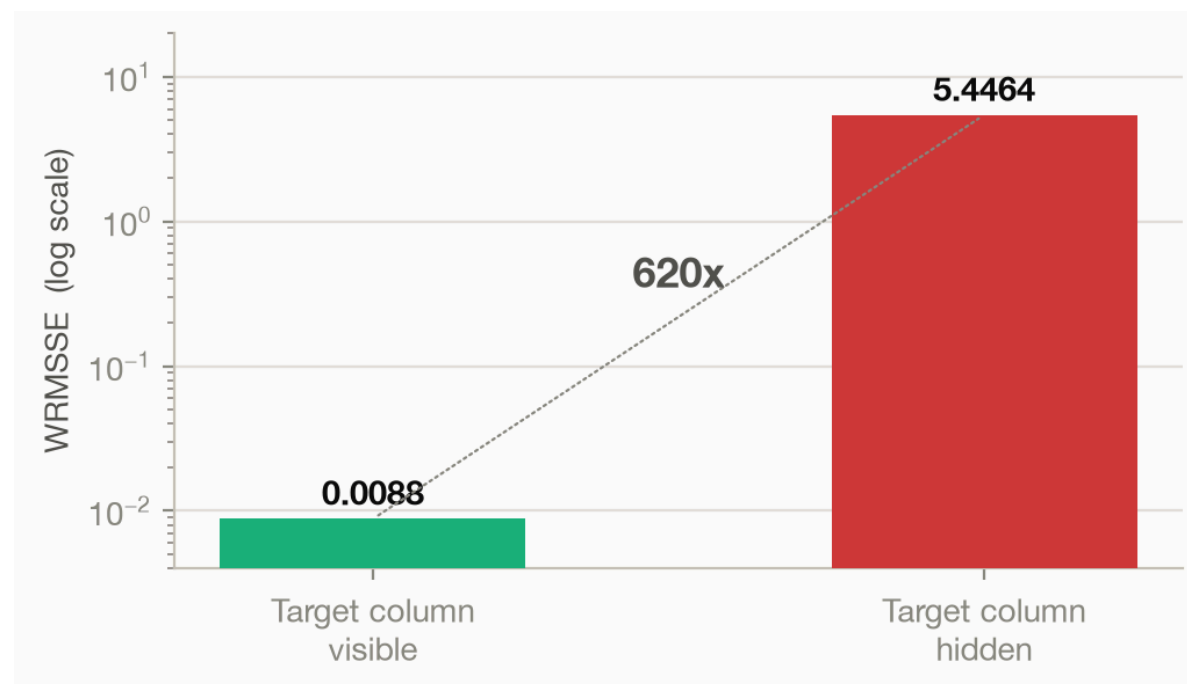
Implementation

A Score That Looked Too Good

LightGBM first scored **0.0088** — near-perfect, and impossible.

The target column had ended up inside the feature set. The model was not forecasting sales, it was reading them. One missing comma in a list of columns to exclude, no error raised.

We scored the same models twice: once with that column visible, once blanked out — which is what happens when you forecast a future that has not happened yet.



The lesson is not “check for leaks.” It is that validation scores cannot catch this alone — the number got *better* as the model got more broken.

Context

Data

Solution

Implementation

What This Means for Stocking Decisions

Ship the baseline

A naive seasonal forecast is currently the most accurate option available, costs nothing to run, and never fails silently. It should be the default until a model genuinely beats it.

Guard every deployment

Any model that goes live is scored against the baseline on the same days. If it loses, it does not ship. This one check would have caught every problem found here.

Know where it is weak

The GRU is competitive on individual items but poor on store and category totals. Aggregate planning and shelf-level replenishment need different treatment.

Honest position: this work does not yet justify replacing existing planning. What it does provide is a scoring harness that makes the next attempt measurable.

Next Steps

1. Train longer

Five epochs was the budget, and the loss was still falling when it stopped. This is the cheapest available gain.

2. Tell it the weekday

The strongest pattern in the data is weekly, and the networks are never told which day it is. They are expected to infer it.

3. Drop the recursion

Predicting all 28 days at once avoids the feedback loop that flattens the forecast. The Seq2Seq model already does this and deserves a proper attempt.

4. Fix the validation split

Hold out a chronological block. The default trailing split removes the most recent data, which is the most useful data there is.

Then: re-score against the baseline. If it still loses, the honest answer stays the same.

Context

Data

Solution

Implementation



Thank you